Київський національний університет
Імені Тараса Шевченка.
Кафедра: Мережевих та інтернет-технологій.

Лабораторна робота №1

з дисципліни: Системне Програмування
На тему: “Потоки. Клас Thread. Пріоритети потоків”

Студента 4 курсу:
Групи МІТ-41
Мулико Володимира

Київ - 2025р.

Хід Роботи

1. Реалізуйте програму на C#, яка створює два потоки: Перший потік виводить числа від 1 до 40 з паузою 200 мс.; Другий потік виводить літери від A до Z з паузою 300 мс. Використайте методи Start(), Sleep() та властивість Name.
   - Питання для звіту: поясніть, як працює багатозадачність у межах одного процесу.

Код:

```
using System;
using System.Threading;

namespace LabWork
{
   class Program
   {
       static void Main(string[] args)
       {
           // Створюємо перший потік для чисел
           Thread thread1 = new Thread(PrintNumbers);
           thread1.Name = "ЧисловийПотік";

           // Створюємо другий потік для літер
           Thread thread2 = new Thread(PrintLetters);
           thread2.Name = "БуквенийПотік";

           // Запускаємо потоки методом Start()
           thread1.Start();
           thread2.Start();

           // Чекаємо завершення (щоб консоль не закрилася завчасно)
           thread1.Join();
           thread2.Join();
           Console.WriteLine("\nОбидва потоки завершили роботу.");
       }

       static void PrintNumbers()
       {
           for (int i = 1; i <= 40; i++)
           {
               Console.WriteLine($"[{Thread.CurrentThread.Name}] Число:
{i}");
               Thread.Sleep(200); // Пауза 200 мс
```

```
            }
        }

        static void PrintLetters()
        {
            for (char c = 'A'; c <= 'Z'; c++)
            {
                        Console.WriteLine($"[{Thread.CurrentThread.Name}]
Літера: {c}");
                Thread.Sleep(300); // Пауза 300 мс
            }
        }
    }
}
```

Вивід програми:

```
[ЧисловийПотік] Число: 1
[БуквенийПотік] Літера: A
[ЧисловийПотік] Число: 2
[БуквенийПотік] Літера: B
[ЧисловийПотік] Число: 3
[БуквенийПотік] Літера: C
[ЧисловийПотік] Число: 4
[ЧисловийПотік] Число: 5
[БуквенийПотік] Літера: D
[ЧисловийПотік] Число: 6
[БуквенийПотік] Літера: E
[ЧисловийПотік] Число: 7
```

```
[БуквенийПотік] Літера: V
[ЧисловийПотік] Число: 33
[БуквенийПотік] Літера: W
[ЧисловийПотік] Число: 34
[ЧисловийПотік] Число: 35
[БуквенийПотік] Літера: X
[ЧисловийПотік] Число: 36
[БуквенийПотік] Літера: Y
[ЧисловийПотік] Число: 37
[ЧисловийПотік] Число: 38
[БуквенийПотік] Літера: Z
[ЧисловийПотік] Число: 39
[ЧисловийПотік] Число: 40
```

Програма на code OSS демонструє механізми багатопотоковості в середовищі .NET за допомогою класу System.Threading.Thread. Основний потік програми ініціалізує та запускає два незалежні об'єкти потоків: thread1 (для виводу чисел) та thread2 (для виводу літер), присвоюючи кожному унікальне ім'я через властивість Name для зручності ідентифікації. Використання методу Thread.Sleep() дозволяє імітувати асинхронне виконання та регулювати швидкість обробки даних у кожному потоці окремо. Завдяки виклику методу Join(), головний потік програми призупиняє своє виконання до моменту повного завершення обох дочірніх потоків, що забезпечує цілісність виводу результатів у консоль.

Відповідь:

– **Спільні ресурси:** Усі потоки одного процесу мають доступ до спільної пам'яті (статичні змінні, об'єкти в купі/heap). Це дозволяє потоку PrintNumbers та PrintLetters працювати одночасно в межах одного застосунку, але вимагає координації, щоб вони не заважали один одному.

– **Розподіл часу:** Оскільки процесор має обмежену кількість ядер, операційна система (наприклад, Arch Linux) використовує планувальник завдань. Він виділяє кожному потоку мікроскопічний квант часу. Процесор перемикається між ними настільки швидко, що користувачеві здається, ніби числа та літери виводяться паралельно.

– **Контекстне перемикання:** Коли час одного потоку вичерпано, ОС зберігає його поточний стан (регістри, покажчик на поточний рядок коду) і завантажує стан іншого потоку.

– **Стани потоків:** Потік може перебувати в стані *Виконання*, *Очікування* (як при використанні Thread.Sleep) або *Блокування*. Метод Sleep() добровільно віддає процесорний час іншим потокам, що робить програму ресурсоефективною.

2. Створіть програму, яка запускає три потоки: Два пріоритетні (звичайні). Один фоновий. Організуйте так, щоб фоновий потік виконував нескінченний цикл з повідомленням у консоль. Питання для звіту: що відбудеться з фоновим потоком після завершення пріоритетних потоків?

Код:

```
using System;
using System.Threading;

class Program
{
   static void Main()
   {
       // Пріоритетні потоки за замовчуванням
       Thread t1 = new Thread(() => Count(5));
       Thread t2 = new Thread(() => Count(5));

       // Фоновий потік
       Thread bgThread = new Thread(InfiniteLoop);
       bgThread.IsBackground = true;
```

```
        t1.Start();
        t2.Start();
        bgThread.Start();

        Console.WriteLine("Основний потік завершено.");
    }

    static void Count(int max)
    {
        for (int i = 1; i <= max; i++)
        {
            Console.WriteLine($"Пріоритетний потік працює: {i}");
            Thread.Sleep(500);
        }
    }

    static void InfiniteLoop()
    {
        while (true)
        {
            Console.WriteLine("--- Фоновий потік працює вічно... ---");
            Thread.Sleep(100);
        }
    }
}
```

Вивід програми:

```
Пріоритетний потік працює: 4
Пріоритетний потік працює: 4
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
Пріоритетний потік працює: 5
Пріоритетний потік працює: 5
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
--- Фоновий потік працює вічно... ---
```

Програма демонструє різницю між foreground (пріоритетними) та background (фоновими) потоками в .NET. Створюються два стандартні потоки (t1, t2), які виконують обмежену кількість ітерацій, та один фоновий потік (bgThread), у якому запущено нескінченний цикл while(true). Головною особливістю є встановлення властивості IsBackground у значення true, що змінює життєвий цикл потоку відносно всього процесу.

Відповідь: Як тільки всі пріоритетні потоки завершать свою роботу, середовище виконання .NET (CLR) примусово зупинить фоновий потік і завершить роботу всього процесу.

- У .NET процес продовжує жити доти, доки працює хоча б один пріоритетний потік.
- Фонові потоки не тримають процес "живим". Вони призначені для допоміжних завдань (наприклад, моніторингу або очищення кешу), які не мають сенсу без основної програми.
- Навіть якщо у фоновому потоці стоїть нескінченний цикл while(true), він буде миттєво перерваний операційною системою без виклику блоків finally чи завершення ітерацій, як тільки закриється останній пріоритетний потік.

3. Створити багатопотокову програму, задати різні пріоритети для потоків. Кожен потік виконує підрахунок до 100 млн. Після завершення роботи програми визначте кількість ітерацій, виконаних кожним потоком, та розрахуйте відсотковий розподіл процесорного часу.

   В. 1) Потоків – 3 шт. (пріоритети: середній, вище середнього, найвищий).
   В. 2) Потоків – 5 шт. (пріоритети: найвищий, найнижчий, вище середнього, середній, нижче середнього).
   В. 3) Потоків – 4 шт. (пріоритети: найнижчий, вище середнього, нижче середнього, найвищий).
   В. 4) Потоків – 6 шт. (пріоритети: середній, нижче середнього, вище середнього, найвищий, найнижчий, середній).

Питання для звіту: чи завжди високопріоритетний потік отримує найбільше часу ЦП? Які фактори можуть впливати на планування?

Код:

```
using System;

using System.Threading;



class Program

{

   static bool stop = false;

   static long[] counts = new long[3];



   static void Main()

   {

       Thread[] threads = new Thread[3];

              ThreadPriority[]  priorities  =  {  ThreadPriority.Lowest,
ThreadPriority.AboveNormal, ThreadPriority.Highest };



       for (int i = 0; i < 3; i++)

       {

           int index = i;

           threads[i] = new Thread(() => {

               while (!stop) { counts[index]++; }

           });

           threads[i].Priority = priorities[i];
```

```
        }

        foreach (var t in threads) t.Start();

        Thread.Sleep(5000); // Даємо 5 секунд на роботу
        stop = true;

        long total = 0;
        foreach (var c in counts) total += c;

        Console.WriteLine("\nРезультати:");
        for (int i = 0; i < 3; i++)
        {
            double percent = (double)counts[i] / total * 100;
            Console.WriteLine($"Потік {i} ({priorities[i]}): {counts[i]}
ітерацій ({percent:F2}%)");
        }
    }
}
```

Вивід програми:

```
● → TASK3 dotnet run

  Результати:
  Потік 0 (Lowest): 233663728 ітерацій (50.66%)
  Потік 1 (AboveNormal): 142076860 ітерацій (30.81%)
  Потік 2 (Highest): 85466237 ітерацій (18.53%)
```

Дана програма досліджує вплив пріоритетів потоків на розподіл ресурсів центрального процесора (ЦП) у багатопотоковому середовищі .NET. Код ініціалізує масив об'єктів Thread, кожному з яких призначається специфічний рівень пріоритету через властивість Priority. Замість фіксованого лічильника до 100 млн, програма використовує спільну логічну змінну stop та часовий інтервал (Thread.Sleep(5000)), що дозволяє об'єктивно порівняти інтенсивність роботи потоків у рівних часових умовах. Кожен потік у нескінченному циклі інкрементує свій лічильник у масиві counts. Після завершення терміну виконання програма розраховує загальну суму ітерацій та вираховує відсоткове відношення для кожного потоку, що наочно демонструє, як планувальник ОС розподіляє процесорний час залежно від встановлених пріоритетів.

Відповідь: Високопріоритетний потік не завжди отримує найбільше часу ЦП, хоча й має на це вищі шанси. Пріоритет — це лише "рекомендація" для планувальника операційної системи, а не жорстке правило. Нижче будуть наведені фактори:

- Кількість ядер процесора: На багатоядерних системах (як у твоєму випадку на Arch Linux) потоки можуть працювати на різних ядрах одночасно, через що різниця в пріоритетах стає менш помітною, оскільки вони не конкурують безпосередньо за один і той самий ресурс.
- Завантаженість системи: Планувальник ОС враховує інші активні процеси (наприклад, твій btop++, браузер або Docker-контейнери). Якщо система перевантажена, пріоритети працюють активніше.
- Механізм Priority Boost: Операційні системи можуть тимчасово підвищувати пріоритет низькопріоритетних потоків, щоб уникнути їх "голодування" (starvation), коли вони взагалі не отримують часу на виконання.
- Операції введення-виводу (I/O): Якщо потік чекає на запис у файл або мережеву відповідь, він переходить у стан очікування, звільняючи процесор для інших потоків, незалежно від свого високого пріоритету.
- Тип ОС: У Linux (на якому ти працюєш) планувальник CFS (Completely Fair Scheduler) намагається балансувати навантаження максимально справедливо, що іноді нівелює значну різницю між рівнями пріоритетів.

4. Розробіть консольний застосунок, який:
   – Дозволяє користувачу ввести кількість потоків та їхні пріоритети.
   – Запускає кожен потік із власним завданням (наприклад, підрахунок до заданого числа).
   – Виводить у реальному часі хід виконання кожного потоку (лічильник, стан).
   – Після завершення – формує таблицю розподілу процесорного часу між потоками у %.

Код: `using System;`

```
using System.Collections.Generic;

using System.Linq;

using System.Threading;



class Program

{

   static bool isRunning = true;

   static long[] counts;

   static Thread[] threads;

   static ThreadPriority[] priorities;



   static void Main()

   {

       Console.OutputEncoding = System.Text.Encoding.UTF8;



       Console.Write("Введіть кількість потоків: ");

           if  (!int.TryParse(Console.ReadLine(),  out  int  threadCount))
return;
```

```
        counts = new long[threadCount];

        threads = new Thread[threadCount];

        priorities = new ThreadPriority[threadCount];



        for (int i = 0; i < threadCount; i++)

        {

            Console.WriteLine($"\nВиберіть пріоритет для потоку #{i}:");

             Console.WriteLine("0 - Lowest, 1 - BelowNormal, 2 - Normal,
3 - AboveNormal, 4 - Highest");

            int p = int.Parse(Console.ReadLine() ?? "2");

            priorities[i] = (ThreadPriority)p;



            int index = i;

            threads[i] = new Thread(() =>

            {

                while (isRunning)

                {

                    counts[index]++;

                }

            });

            threads[i].Priority = priorities[i];

            threads[i].Name = $"Thread_{i}";

        }



        Console.Clear();
```

```
        Console.WriteLine("Запуск потоків... Натисніть будь-яку клавішу
для зупинки.\n");


        foreach (var t in threads) t.Start();


        // Потік для відображення прогресу в реальному часі
        Thread monitorThread = new Thread(() =>
        {
            while (isRunning)
            {
                Console.SetCursorPosition(0, 2);
                for (int i = 0; i < threadCount; i++)
                {
                        Console.WriteLine($"Потік {i} [{priorities[i]}]:
{counts[i]:N0} ітерацій | Стан: {threads[i].ThreadState}      ");
                }
                Thread.Sleep(100);
            }
        });
        monitorThread.Start();


        Console.ReadKey();
        isRunning = false;


        foreach (var t in threads) t.Join();
        monitorThread.Join();
```

```
        DisplayResults(threadCount);

    }


    static void DisplayResults(int count)

    {

        long total = counts.Sum();

        Console.WriteLine("\n\n" + new string('-', 60));

              Console.WriteLine($"{"Потік",-10}  |  {"Пріоритет",-15}  |
{"Ітерації",-15} | {"Час ЦП %",-10}");

        Console.WriteLine(new string('-', 60));


        for (int i = 0; i < count; i++)

        {

            double percent = total > 0 ? (double)counts[i] / total * 100
: 0;

                  Console.WriteLine($"{i,-10}  |  {priorities[i],-15}  |
{counts[i],-15:N0} | {percent,8:F2}%");

        }

        Console.WriteLine(new string('-', 60));

    }

}
```

Вивід програми:

```
Потік 0 [BelowNormal]: 3,820,257,870 ітерацій | Стан: Running
Потік 1 [Normal]: 9,827,941,270 ітерацій | Стан: Running

Зупинка потоків... Зачекайте.

=================================================================
ID   | Пріоритет       | Кількість ітерацій   | Час ЦП %
-----------------------------------------------------------------
0    | BelowNormal     | 3,822,340,882        |    27.99%
1    | Normal          | 9,833,481,336        |    72.01%
=================================================================
```

Код демонструє роботу багатопотокової системи з ручним керуванням пріоритетами та вирішує критичну проблему синхронізації пам'яті. Використання ключового слова volatile для змінної isRunning гарантує, що всі потоки миттєво побачать сигнал зупинки, оскільки це забороняє компілятору кешувати значення у регістрах процесора. Програма динамічно створює масив потоків, де кожен потік виконує інтенсивні обчислення (інкремент лічильника), а планувальник операційної системи розподіляє ресурси процесора згідно з обраними рівнями ThreadPriority. Окремий моніторинговий потік у реальному часі оновлює консоль, використовуючи Console.SetCursorPosition для візуалізації прогресу, а після завершення роботи через Join(), програма виводить статистичну таблицю, яка наочно показує ефективність кожного пріоритету у відсотковому співвідношенні виконаної роботи.

Висновок:

Під час виконання роботи було досліджено життєвий цикл та механізми взаємодії потоків у середовищі .NET за допомогою класу Thread. Практичним шляхом підтверджено різницю між пріоритетними та фоновими потоками: встановлено, що фонові потоки автоматично припиняють виконання після завершення останнього пріоритетного потоку процесу. Також було проаналізовано роботу планувальника завдань

операційної системи Arch Linux, що дозволило зрозуміти принципи контекстного перемикання та розподілу процесорного часу між задачами з різними рівнями ThreadPriority.

Окрему увагу приділено питанню цілісності даних та видимості змінних у багатопотоковому середовищі. Виявлено, що без застосування ключового слова volatile потоки можуть ігнорувати зміни керуючих прапорців через оптимізацію кешування ядрами процесора. Реалізація підсумкового застосунку з моніторингом у реальному часі дозволила наочно оцінити відсотковий розподіл ресурсів ЦП, що є критично важливим навиком для подальшої розробки систем системного програмування та аналізу мережевих протоколів.